

六、避免死锁的银行家算法

6.1 实验目的

理解银行家算法避免死锁的工作原理
掌握银行家算法避免死锁的工作过程
掌握系统安全性检测算法

6.2 预备知识

一、避免死锁

避免死锁属于事先预防的策略，但并不是事先采取某种限制措施，破坏产生死锁的必要条件，而是在资源动态分配过程中，防止系统进入不安全状态，以避免发生死锁，此方法所施加的限制条件较弱，可能获得较好的系统性能，目前常用此方法来避免发生死锁。

1、安全状态

安全状态指系统能按某种进程顺序($P_1, P_2, \dots, P_n$)为每个进程 P_i 分配其所需资源，直至满足每个进程对资源的最大需求，使每个进程都可顺利地完成。

此时，称 $(P_1, P_2, \dots, P_n)$ 为安全序列。

如果系统无法找到这样一个安全序列，则称系统处于不安全状态。避免死锁的实质在于：在进行资源分配时，应使系统不进入不安全状态。

在避免死锁的方法中，允许进程动态地申请资源，但系统在进行资源分配之前，应先计算此次资源分配的安全性。若此次分配不会导致系统进入不安全状态，则将资源分配给进程；否则，让进程进入等待状态。

2、安全状态与不安全状态之例

假定系统中三个进程 P_1 、 P_2 和 P_3 ，共有 12 台磁带机。进程 P_1 总共要求 10 台磁带机， P_2 和 P_3 分别要求 4 台和 9 台。假设在 T_0 时刻，进程 P_1 、 P_2 和 P_3 已分别获得 5 台、2 台和 2 台磁带机，尚有 3 台空闲未分配。如下表所示。系统此时处于安全状态。因为存在一个安全序列： $P_2 \rightarrow P_1 \rightarrow P_3$ ，使得每一个进程都能顺利结束。

进 程	最大需求	已 分 配	可 用
P_1	10	5	3
P_2	4	2	
P_3	9	2	

若此时 P_1 申请一台磁带机，或者 P_2 申请分配磁带机。分配后，系统仍然处于安全状态下，所以可以满足它们的资源分配请求。

若 P_1 申请 2 台以上的磁带机，或者 P_3 申请分配 1 台以上的磁带机，若进行分配，会使系统进入不安全状态，因此不能满足它们的资源分配请求。

二、银行家算法

银行家算法顾名思义是来源于银行的借贷业务，一定数量的本金要满足多个客户的借贷周转，为了防止银行家资金无法周转而倒闭，对每一笔贷款，必须考察其是否能限期归还。在操作系统中研究资源分配策略时也有类似问题，系统中有限的资源要供多个进程使用，必

须保证得到的资源的进程能在有限的时间内归还资源，以供其它进程使用资源。如果资源分配不当，就会发生进程循环等待资源，则进程都无法继续执行下去的死锁现象。所以，操作系统中的银行家算法通过测试系统在分配资源后，是否仍然处于安全状态，从而决定是否进行资源分配，以避免出现死锁的算法。

1、银行家算法中的数据结构

(1) 可利用资源向量 Available

一个含有 m 个元素的数组，其中的每一个元素代表一类可利用的资源数目，其初始值是系统中所配置的该类全部可用资源的数目，其数值随该类资源的分配和回收而动态地改变。

如果 $Available[j]=K$ ，则表示系统中现有 R_j 类资源 K 个。

(2) 最大需求矩阵 Max

一个 $n \times m$ 的矩阵，它定义了系统内 n 个进程中的每一个进程对 m 类资源的最大需求。如果 $Max[i,j]=K$ ，则表示进程 P_i 需要 R_j 类资源的最大数目为 K 。

(3) 分配矩阵 Allocation

一个 $n \times m$ 的矩阵，它定义了系统中每一类资源当前已分配给每一进程的资源数目。如果 $Allocation[i,j]=K$ ，则表示进程 P_i 当前已分得 R_j 类资源的数目为 K 。

(4) 需求矩阵 Need

一个 $n \times m$ 的矩阵，用以表示每一个进程尚需的各类资源数。如果 $Need[i,j]=K$ ，则表示进程 P_i 还需要 K 个 R_j 类资源，方能完成其任务。

上述三个矩阵间存在下述关系：

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$

2、银行家算法过程

(1)如果 $Request[i] \leq Need[i, j]$ ，便转向步骤(2)；否则认为出错，因为它所需要的资源数已超过它所宣布的最大值。

(2)如果 $Request[i] \leq Available[j]$ ，便转向步骤(3)；否则，表示尚无足够资源， P_i 须等待。

(3)系统试探着把资源分配给进程 P_i ，并修改下面数据结构中的数值：

$$Available[j] = Available[j] - Request[i][j];$$

$$Allocation[i, j] = Allocation[i, j] + Request[i][j];$$

$$Need[i, j] = Need[i, j] - Request[i][j];$$

(4)系统执行安全性算法，检查此次资源分配后系统是否处于安全状态。若安全，才正式将资源分配给进程 P_i ，以完成本次分配；否则，将本次的试探分配作废，恢复原来的资源分配状态，让进程 P_i 等待。

3、安全性算法

系统所执行的安全性算法可描述如下：

(1) 设置两个向量

① 工作向量 **Work**，它表示系统可提供给进程继续运行所需的各类资源数目，它含有 m 个元素，在执行安全算法开始时， $Work:=Available$ 。

② **Finish**，它表示系统是否有足够的资源分配给进程，使之运行完成。开始时先做 $Finish[i] = false$ ；当有足够资源分配给进程时，再令 $Finish[i]:=true$ 。

(2) 从进程集合中找到一个能满足下述条件的进程：

① $Finish[i] = false$ ；

② $Need[i, j] \leq Work[j]$ ；若找到，执行步骤(3)，否则，执行步骤(4)。

(3) 当进程 P_i 获得资源后，可顺利执行，直至完成，并释放出分配给它的资源，故应执行：

$Work[j] = Work[j] + Allocation[i,j];$

$Finish[i] = true;$

go to step (2);

(4) 如果所有进程的 $Finish[i] = true$ 都满足，则表示系统处于安全状态；否则，系统处于不安全状态。

6.3 实验内容

使用 C 语言设计并实现银行家算法模拟程序。

6.4 实验指导

1、定义相关数据结构

(1)可用资源向量 Available， m 维，表示未分配的各种可用资源数量。

(2)工作向量 Work， m 维，表示系统可提供给进程继续运行所需的各类资源数目。

(3)完成向量 Finish， m 维，表示系统是否有足够的资源分配给进程，使之运行完成。

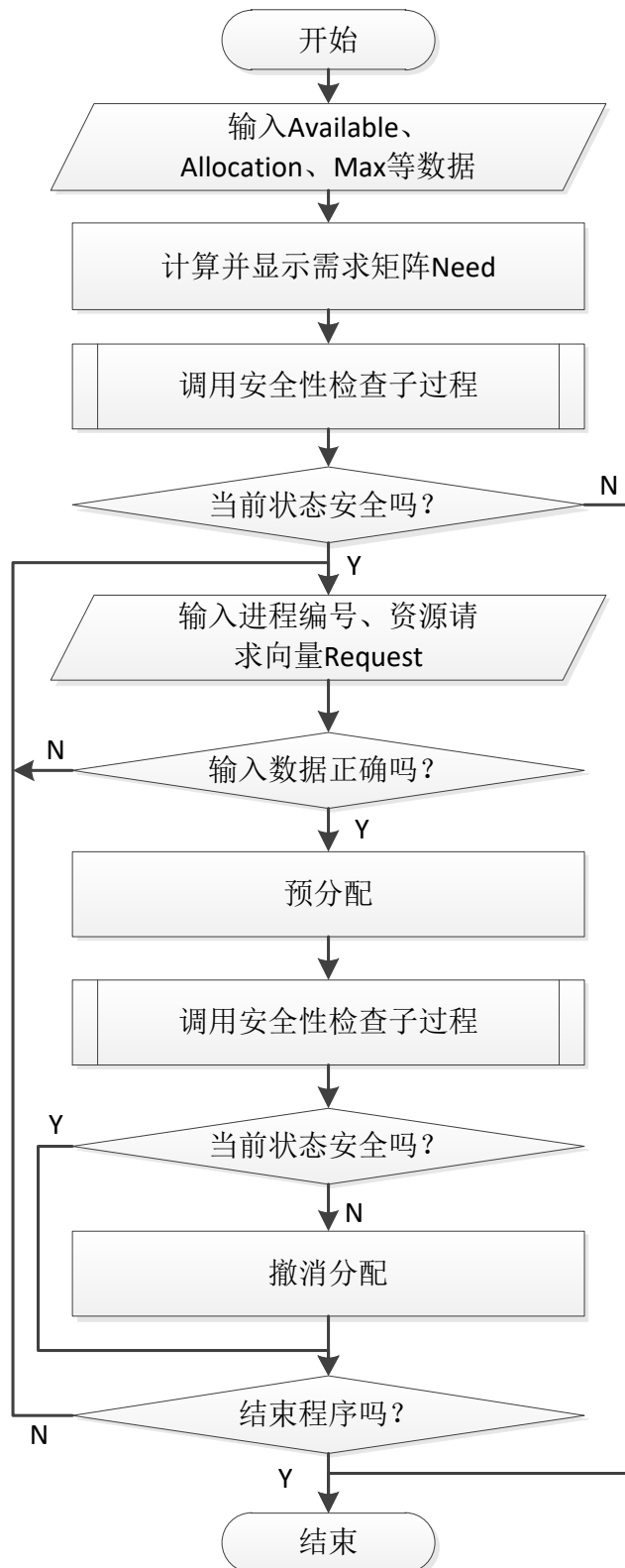
(4)请求向量 Request， m 维，表示进程向系统提交的资源分配申请。

(5)最大需求矩阵 Max， $n \times m$ 矩阵，表示 n 个进程对 m 类资源的最大需求。

(6)分配矩阵 Allocation， $n \times m$ 矩阵，表示 n 个进程已分配的各种资源数。

(7)需求矩阵 Need， $n \times m$ 的矩阵，表示每一个进程尚需的各类资源数。

2、银行家算法流程



3、安全性算法流程

